Probability, R and Computational Finance

Martin Summer

2021-09-30

Introduction

> "What I hear, I forget; What I see, I remember; What I do, I understand"
>
> — Confucius, 551-479 BC

Our lectures are going to teach you basic concepts of probability every finance practitioner needs to know and be familiar with. Because you will need probability as a tool for thinking about and solving finance problems with computational methods, we will use computational finance as the context within which we develop the concepts and ideas of probability. But we will also make much use of the computer in this course: Our approach to teaching you basic concepts of probability is based on understanding concepts by building them with your own hands.

But how can we build abstract concepts such as probability, random phenomena and chance by our own hands? How can we "touch" such things like an idea? While probability is a mathematical theory, it gains practical value and an intuitive meaning in connection with real or conceptual experiments such as, the future change in a stock price index, the future value of a portfolio of securities, the chance that a creditor is not going to be able to pay back your loan.

Many of these experiments we can nowadays simulate on the computer. We can can construct, build and simulate a huge variety of random phenomena. We can - for instance - implement models of random fluctuations of asset prices. We can model financial risks and contemplate possible future scenarios through simulation.

We can approach our understanding of abstract concepts by building them with our own hands on the computer. This is the approach to teaching you probability in this course.

There are many ways to build virtual objects and to run simulations to manipulate them. For this we will need a programming language. The language we choose for this course is R and the integrated development environment RStudio. This is one of the main languages used in data analysis, statistics and data science and is widely used in industry and academia. It will be our tool to do probability in this course.

Downloading and installing R

So lets start by downloading and installing R first. R is an open source project maintained by an international team of developers. The software is made available through a website called the comprehensive R archive network (http://cran.r-project.org).

At the top of this website, in a box named "Download and install R" you will find three links for downloading R. Choose the link that describes your operating system, Windows, Mac or Linux. These links will lead you to the necessary information you need to install a current version of R. The easiest install option is to install R from precompiled binaries. There is also the option to built R from source on all operationg systems if you have the tools and the expertise to do so. R also comes both in a 32-bit and a 64-bit version. It does not make a substantial difference which version you use. 64-bit versions can handle larger files and data sets with fewer memory management problems.

Downloading and installing RStudio

RStudio is an application that helps you write and develop R code. It makes using R much easier for you than using it in isolation. The interface of RStudio looks the same across all operating systems.

You can download RStudio for free from https://www.rstudio.com/products/rstudio/. Select the box RStudio Desktop and follow the download instructions. RStudio Desktop is free. Note that you need to have a version of R installed to use RStudio.

If you have successfully installed R and RStudio, we are ready to start.

Empirical background and some terminology

If you chose to study Finance it should be natural for you to study probability. After all, Finance studies the allocation and pricing of money across time. You can safe money to use it later for purchasing goods and services you can borrow money to invest today and repay in the future from the revenues of your project. But the future is uncertain and we do not know today how it will look like. In Finance we unavoidably have to deal with risk and uncertainty. The most fundamental concept we have available to deal with risk and uncertainty and to think about and analyze it is probability. Therefore every serious student of Finance has to study probability early on.

Terminology

Probability is a mathematical theory. This theory gets practical value and intuitive meaning in connection with real or conceptual experiments such as selecting a random sample of stocks and observing their most extreme price drop over a given period of time or observing the frequency of loan defaults. In the early days of probability when some conceptual ideas about which we are going to learn in this course were first developed the conceptual experiments considered by the pioneers of probability where games of chance involving rolls of dies, or spins of a wheel as well as reflections on placing bets on the outcomes of such games. While we will stick mainly to our main context - computational finance - some of these chance games are so intuitive and insightful that we will occasionally in the beginning of the course also use some of them for illustrating concepts and playing with them.

All of the examples and descriptions given above are, however, rather vague. We have to be more precise what we mean by an experiment or observation.

When we use the term (random) *experiment* we mean a process leading to an uncertain outcome. To make this notion precise and useful we need to agree what we mean by uncertain outcomes.

Take the simple example of considering whether the price of a stock is going to rise or fall at the next day. In a practical situations the outcome of a move in the stock price can be that it goes up or down but it could in principle also stay the same. Still when we think about the experiment of observing the stock price tomorrow in many applications in Finance we usually agree *on the outset* that *up* and *down* are the only possible outcomes of this experiment. By such an agreement we have pinned down the possible outcomes.

The collection of all possible outcomes of an experiment is called the *sample space*. While the sample space of the experiment of observing tomorrow's stock price is an idealization it is exactly this idealization which simplifies the theory without affecting its applicability.

Idealizations of this kind are standard in probability. It is - for example - impossible to know how high the price of a stock might become. Is there a maximal price beyond which the price can not possibly hike? Many of us would hesitate to claim that the price might rise without bound. Yet many models in applied Finance are based on such an assumption. The models allow arbitrary price hikes but with arbitrary small probability as the price gets higher and higher. Practically it does not make sense to believe that a security price can become arbitrarily high. The use of arbitrarily small probabilities in a financial model might seem absurd but it does no practical harm and makes the model simple and convenient to use. Moreover, if we seriously introduced an upper bound on a security price at x it would be also awkward to assume that it is impossible that it could be just a cent higher, an assumption equally unappealing than assuming it can get in principle arbitrarily high.

When we talk of a *basic outcome* we mean the possible outcome of a random experiment. An *event* is a subset of basic outcomes. Any event which consists of a single outcome in the sample space is called a *simple event*.

Probability is a measure of how likely an event of an experiment is. Thus when we talk about probability in a precise and meaningful sense we can only do so in relation to a given sample space or to a certain conceptual experiment.

Thus before we can begin to seriously talk about probability we need to develop a good understanding of experiments, sample spaces, basic outcomes and events. In the spirit of this course we will develop this understanding by building these concepts using R. Thus let us develop examples of random experiments, sample spaces, basic outcomes and events in parallel with making our first steps in R. R will be our tool which will allow us to "touch" probability concepts.

As we proceed through our course we will develop a broader and deeper understanding of this language together with the probability concepts.

Rolling a die: Constructing a random experiment, a sample space and events using R

Lets start with a classic and old example of a probability model: Rolling a six sided die. The rolling of the die is one of the non-finance examples we mentioned before. It is so useful that we will come back to it once in a while in the beginning of the course. Here its usefulness is enhanced because it allows us in an easy way to quickly learn some powerful R concepts and techniques. We want to roll this die not physically but on our computer using R and RStudio as our tool. How can we do this?

The R User Interface

Before we can ask our tool to do anything for us, we need to know how to talk to it. In our case RStudio allows us to talk to our computer. It works like any other application. When you launch RStudio on your computer, you will see a screen looking like this:

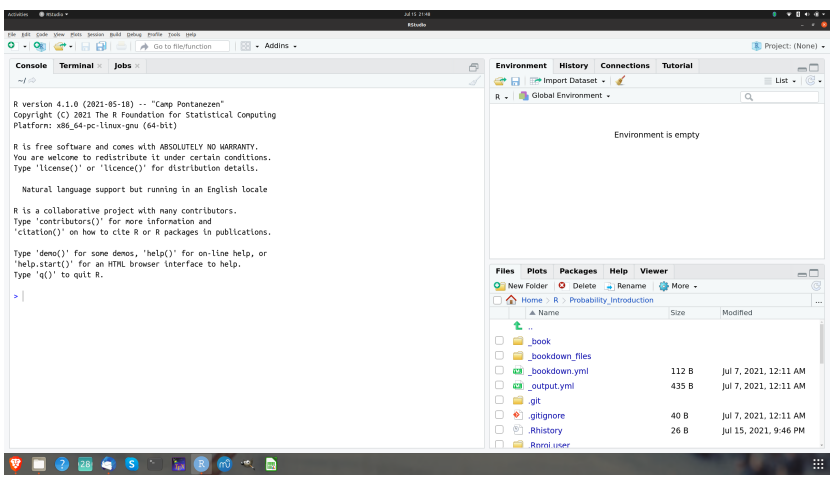


Figure 1: The RStudio startup screen

In this picture you see a screenshot of my RStudio screen. Interacting with the app is easy. You type commands via your keyboard at the prompt, which is the > symbol. You can see it in the left pane in the screenshot. You send the command to the computer by pressing enter. After you have pressed enter, RStudio sends the command to R and displays the result of your command with a new prompt to enter new commands, like this:

```
> 1 + 1
[1] 2
>
```

The `[1]` means that the line begins with the first value of your result. For example, if you enter the command `20:60` which means in the R language, "list all the integers from 20 to 60," and you then press return, you will get an output like this:

```
[1] 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 4
[34] 53 54 55 56 57 58 59 60
```

meaning that 20 is the first value displayed in your result. Then there is a line break because not all values can be displayed on the same line and R tells you that `53` is the 34-th value of the result.

The colon operator `:` is a very useful function in R which we will often need. It allows us to create sequences of every integer between two given integers.

R needs a complete command to be able to execute it, when the return key is pressed. If a command is incomplete, like for instance `5*`, where we enter 5, the multiplication operator but no second factor to multiply 5 with R will show:

```
> 5*
+
```

The `+` sign means that R needs more input to complete the expression. So if we type the missing factor after the `+` prompt, say 4 and then press the return key again, we will get the result

```
> 5*
+ 4
[1] 20
```

If you type a command that R does not understand, you will be returned an error message. Don't worry if you see an error message. It just is a way the computer tells you that he does not understand what you want him to do.

For instance, if you type `5%3` you will get an error message like this

```
> 5%3
Error: unexpected input in "5%3"
>
```

Sometimes it is obvious why a mistake occurred. In this case, that R just does not know what to do with the symbol `%`. It has no meaning in this context. Sometimes it is not so obvious what the error message actually means and what you might do about it. A useful strategy in this case is to type the error message into a search engine and see what you can find. The chance is very high that others encountered the same problem before you and got helpful advice how to fix it from other users on the internet. One site, we find particularly helpful for all kinds of questions related to R and R programming is https://stackoverflow.com/. Try it at the next opportunity.

Now with this basic knowledge, we can already make the first step to create a die on the computer using R. If you think of a physical die, the essential thing

that matters are the points on its six sides. If you throw the dice it will usually land on one of these sides and the side which is on the upper side of the die shows the number of points. The colon operator `:` gives us a way to create a group of numbers from 1 to 6. R gives us the result as a one dimensional set of numbers.

```
> 1:6
[1] 1 2 3 4 5 6
>
```

Lets use these first steps in R to recap the probability concepts we have learned using this example of the six sided die: A *basic outcome* of rolling a six-sided die is for example `6` if the upper side after rolling the die happens to be the side with 6 points. The *sample space* of the experiment of rolling a six-sided die is the set $\mathcal{S} = \{1, 2, 3, 4, 5, 6\}$. In probability theory we often use the symbol $\mathcal{S}$ or S for sample space. In many probability texts the sample space is also often denoted by the symbol Ω the greek letter for (big) Omega. A *random experiment* in this example is the rolling of the die. The outcome is uncertain but once the die is rolled the outcome can be determined precisely. Finally we can also illustrate in the example the concept of an *event*: For example, the event, that the die will show less than 4 points is $A = \{1, 2, 3\}$. We use here the symbol A to denote this event and use set notation to describe the set of basic outcomes that make up the event "number of points smaller than 4." The event that the outcome is that the point shows 10 points is the empty set $B = \emptyset$. The symbol $\emptyset$ comes from set theory and means the set containing no elements. This event can contain no elements because we can not get a score of 10 by rolling a six sided die.

Objects

You can save data in R by storing them in *objects*. An object is a name, you can choose yourself to store data. For example, if you choose to store the value `6` in an object called `point_six`, you would type:

```
> point_six <- 6
>
```

at the prompt. R will the store the value 6 in the object called `point_six`, which you can use to refer to the value. If you type the name of your object at the prompt, R will distplay the value you have assigned. A usful key combination for typing the assignment operator `<-` is to use the key combination `ALT _`. At the R prompt R will automatically print an assignment operator.

```
> point_six
[1] 6
>
```

Now you can use the name of the object to refer to its value. For instance, you could divide `point_six`by `2`and get a meaningful result

```
> point_six/2
[1] 3
>
```

Now to make our `die` more tangible and useful, let us store it in an R object by typing the following command at the prompt. This command creates an object with name `die` and assign the vector `1,2,3,4,5, 6` to it.

```
> die <- 1:6
>
```

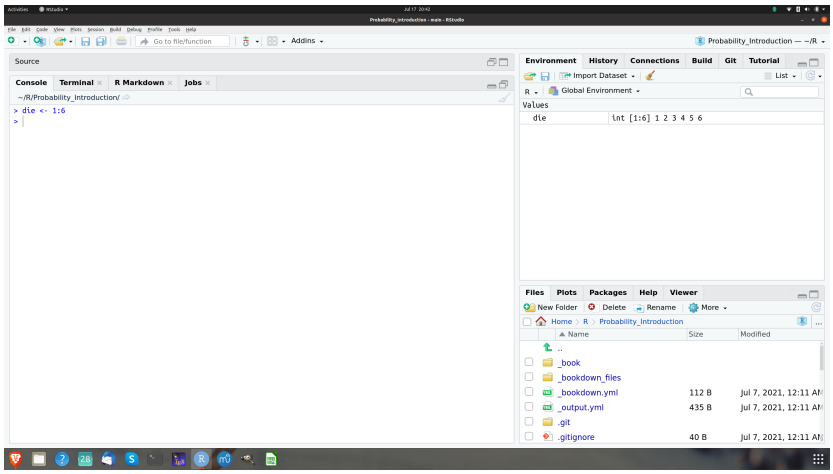


Figure 2: The RStudio Environment pane keeps track of the objects you have created

You can now see in the right upper Environment pane that R shows you that there is an object with the name `die` that it consists of integers `1,2,3,4,5`. As you create more objects they will be stored in the Environment pane and are ready for your reference, unless you delete them. You can remove or delete an object by typing

```
> rm(object)
>
```

or by assigning the value `die <- NULL` which would also remove the object from your environment or workspace.

You can name your objects almost anything with a few exceptions. An object name must not start with a number. There are some special symbols which can also not be used in object names, like `^, !, $, @, +, -, /, *`. Note that R is case sensitive and distinguishes small and big letters. If you assign a new value for an object you have already created, R will overwrite the object without warning.

You can see which objects are currently created and available for you in the Environment pane of your session of by typing `ls()`.

Before we learn how we can actually roll our die and perform a random experiment with it, let us briefly use the opportunity to explain a few things about how R does computations. We have already explained that we can use the object name to refer to the value. So for instance if we type

```
> die*die
[1]  1  4  9 16 25 36
>
```

This might irritate some of you because we have called the object a vector. In linear algebra multiplication of vectors is only allowed if there is an inner product. What happens here, if we use `*` the multiplication operator is that R does an element-wise multiplication of the six numbers of our die. Of course R allows to take an inner product as well, but this needs a different operator. To compute an inner product, we would type

```
> die %*% die
     [,1]
[1,]   91
```

Now R displays the result as a vectors with one row and one column, which is denoted in the output by `[ , 1]` for the column and `[1, ]` for the row. We will learn later more about the use and the meaning of this notation in R.

The elementwise execution R usually uses also means that when you, for example type

```
> die - 1
[1] 0 1 2 3 4 5
>
```

R would subtract `1` from every component in the vector `die`.

Another specific behavior of R, you need to know about is called "recycling." If you give R two vectors of different length in an operation, R will repeat the shorter vector as long as it is of equal length with the longer one. For example, if you have:

```
> die + 1:2
[1] 2 4 4 6 6 8
>
```

you see that R adds 1 to 1 and 2 to 2 and then starts over again by adding 1 to 3 and 2 to 4 and then starts over once again by adding 1 to 5 and 2 to 6. If the longer vectors is not a multiple of the shorter one, R recycles but the cuts off. It will give you a warning though.

```
> die + 1:4
[1] 2 4 6 8 6 8
```

```
Warning message:
In die + 1:4 :
  longer object length is not a multiple of shorter object length
>
```

While this might seem awkward to some of you, we will see that for data manipulation element-wise execution is often extremely useful. It allows to manipulate groups of values in a systematic way.

Functions

R contains many functions which we can use to manipulate data and compute things. The syntax for using a function is very simple: You type the function name and put the value of the function argument in parentheses. Here we use for illustrations the function of the square root `sqrt()` and the function for rounding a number `round()`.

```
> sqrt(4)
[1] 2
> round(3.1415)
[1] 3
>
```

The data you write in the parentheses are called the fuction arguments. Arguments can be all sorts of things: raw data, R objects, results from other functions. If functions are nested, R evaluates the innermost function first and then goes on to the outer functions. To see examples of all these instances you can take

```
> mean(1:6)
[1] 3.5
> mean(die)
[1] 3.5
> round(mean(die))
[1] 4
>
```

for example.

For simulating random experiments, R has the function `sample()`. With this function we can roll our die on the computer and conduct actual random experiments. The function takes as arguments a vector names `x` and a number named `size`. `sample` will return `size` elements randomly chosen from the vector `x`. Lets say:

```
> sample(x = 1:4, size = 2)
[1] 4 1
>
```

In this case `sample` has chosen `4,1` from the vector `x = (1,2,3,4)`. If we want to roll the die in our computer we can thus pass the die as an argument to `sample` and use the number `1` for the `size` argument. Lets do a few rolls with our die

```
> sample(x = die, size = 1)
[1] 3
> sample(x = die, size = 1)
[1] 2
> sample(x = die, size = 1)
[1] 3
> sample(x = die, size = 1)
[1] 5
> sample(x = die, size = 1)
[1] 1
> sample(x = die, size = 1)
[1] 5
>
```

R functions can have many arguments, but they need to be separated by a comma.

Every argument in every function has a name. We specify which data are assigned to the arguments by setting the name equal to the data. Names help us to avaoid passing the wrong data and thereby mixing up things or committing errors. But using names is not necessary. If we just wrote

```
> sample(die, 1)
[1] 5
>
```

R would also know what to do. It is not always clear which names to use for a function. If you are not sure, you can use the function `args` to look it up. Here we take the function `round` as one example.

```
> args(round)
function (x, digits = 0)
NULL
>
```

Note that the `digits` argument in the `round` function is already set to `0`. Frequently R functions come with optional arguments. These arguments are optional because the come with a default value, which is `0` in case of the round function.

We recommend that you write out argument names as a rule. It gives clearer code and avoids errors. If you don't write argument names, R matches your values to the arguments of the function by order.

Writing your own functions

Now we are ready to write our own function to roll the die in our computer. Each function in R has the same elements: A name, a function body of code and a set of arguments. To write your own function, you have to write up all of these parts and save them in an R object. The syntax is:

```
my_function <- function() {}
```

The name here is `my_function`, next comes the expression `function()` which needs to be assigned. The names of the function arguments have to be written between the parentheses. Then we have to write the actual code within the braces `{}`.

To do this for the die, lets write a function names `roll_die`.

```
> roll_die <- function(){die <- 1:6
                         sample(die, size = 1)}
```

Now we can roll our die for a few times to show how the function works

```
> roll_die()
[1] 4
> roll_die()
[1] 1
> roll_die()
[1] 5
> roll_die()
[1] 4
> roll_die()
[1] 6
>
```

A final remark in the `sample`function is in place here. If we look at the argunments of sample using the `args` function we see

```
> args(sample)
function (x, size, replace = FALSE, prob = NULL)
NULL
>
```

Lets do not discuss all the details of this output but concentrate for a moment on the `replace` argument. What does this mean? As we saw previously we can use the sample function to model the rolling of our die. If we set the `size` argument to `1` we get the roll of one die. If we set the `size` argument to `n`, we would simulate the rolling of `n` dies. But now the `replace` argument becomes crucial. As we can see in the output of the `args` function `replace` has a default value `FALSE`. This is a logical argument. It tells R, for example, that if we set `size = 2`, meaning that two dice are rolled, if the first dice shows, say a value of 3, the second die cannot show 3 as well. This is clearly not what we have in

mind when we model the rolling of 2 dice. It should be possible that both dies show the same value. To enable this behavior of the sample function, we have to change the default value of the `replace` argument to `TRUE`. Then R chooses a random draw from all of the six possible values for all dice rolled.

Congratulations. You have written your first R function for conducting a simple random experiment. Think of the parentheses as a trigger that tells R to run the function. If you omit the trigger R just prints the body of the function. When you run a function, all the code in the function body is executed and R returns the result of the last line of code. If the last line of code does not return a value neither will R.

Arguments

Imagine we remove the first line of code in our function body and changed the name die in the sample function to ball.

```
roll_die2 <- function(){sample(ball, size = 1)}
```

If we call the function now, we will get an error

```
> roll_die2()
Error in sample(ball, size = 1) : object 'ball' not found
```

We could supply ball when we call `roll_die2` if we make ball an argument of the function. Lets do this:

```
> roll_die2 <- function(ball){sample(ball, size = 1)}
>
```

Now the function will work as long as we supply `ball` when we call the function.

```
> roll_die2(ball = 1:6)
[1] 2
>
```

Note that we still get an error, if we forget to supply ball argument

```
> roll_die2()
Error in sample(ball, size = 1) :
  argument "ball" is missing, with no default
```

This could be avoided if we give the function a default argument

```
> roll_die2 <- function(ball= 1:6){sample(ball, size = 1)}
> roll_die2()
[1] 4
>
```

Scripts

So far we have worked by interacting with the console. But what if you want to edit your functions? It would be much easier, if you could use a draft of your code and work form there. This can be done by using a *script*.

You create a script by going to `File > New File > R script` in the menu bar of RStudio. Using scripts is the standard way to write code in R. It not only helps you to keep track of your code, save it and edit it later. It also makes your work reproducible. You can edit and proofread your code and share it with others. To save your script go to `File > Save As` in the menu bar.

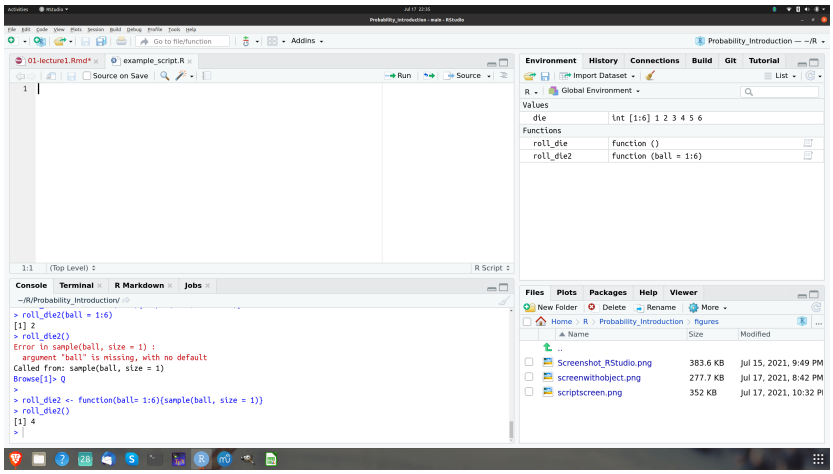


Figure 3: The RStudio Script

RStudio has many useful features to help you work with scripts. You can for instance automatically execute a line in a code by using the run button. You can also execute sections of code or the entire script. The entire script is executed by running the Source button. For all these commands there are key short cuts which you will learn as you work more with RStudio and R.

Using packages and finding Help

We have now a function which we can use to simulate the rolling of a die, `roll_die()`. If the die is fair it should be the case that if we roll the die often, all numbers should occur about equally often. The die should not be weighted in favor of a particular value.

One way to learn whether our die is fair are *repetition* and *visualization*. These are tools we will need all the time, when working with data and when doing probability. While R has many useful functions, one of the great powers is that R is constantly extended by a huge community of users by providing packages.

Packages are add on functions, which will not be available when you install R. They need to be installed and loaded before you can use them. Since packages are such a powerful tool in R we need to introduce what they are and how to use them in the beginning.

Packages

There are many visualization tools in R that come with the basic installation. Since the point we want to make here is about packages, we will use a visualization function which is part of the package *ggplot2*, a very popular package for making all kinds of graphs. There are many additional functions provided with this package. For the point we want to make here we will use one of them, called `qplot`.

Since `qplot` is a function in the package *ggplot2*, we first need to install this package. To install a package you need to be connected to the internet. If you have internet connection go to the command line and run at the command line

```
> install.packages("ggplot2")
```

R will now download the package via the internet and install it. It will display various messages in the console during this process. Don't worry if you don not know what they exactly mean and don't panic that they are displayed in red. All packages can be installed like this. You have just to enter the correct name in the function `install.packages()`. The lower left pane in the RStudio software alternatively provides a tab called **Packages**. Using this tab, you can also install R packages interactively by clicking the install button on the upper left corner of the Packages tab.

After installation the package is on our hard-drive but it can not yet be used. To use the package it has to be laoded. This is done with the command `library`. To load the *ggplot2* package we type

```
> library("ggplot2")
```

and hit the return key. Many things could be said about the R package system and you will learn it in more detail as we go along in our course. For the moment the most important thing to remember now is that a package needs to be newly loaded whenever you want to use it in a new R session.

To check whether our dies is fair, we need to roll it many times. R provides a function, that does this for us. This function is called `replicate` and provides an easy way to repeat a command many times. The number of times we want to repeat something is given as an argument to the `replicate` function.

Now lets roll our die 100 times and save the result in an object we call `rolls`:

```
rolls <-replicate(100, roll_die())
```

We now use the "plot"' function from the *ggplot2* library to make a quick visualization.

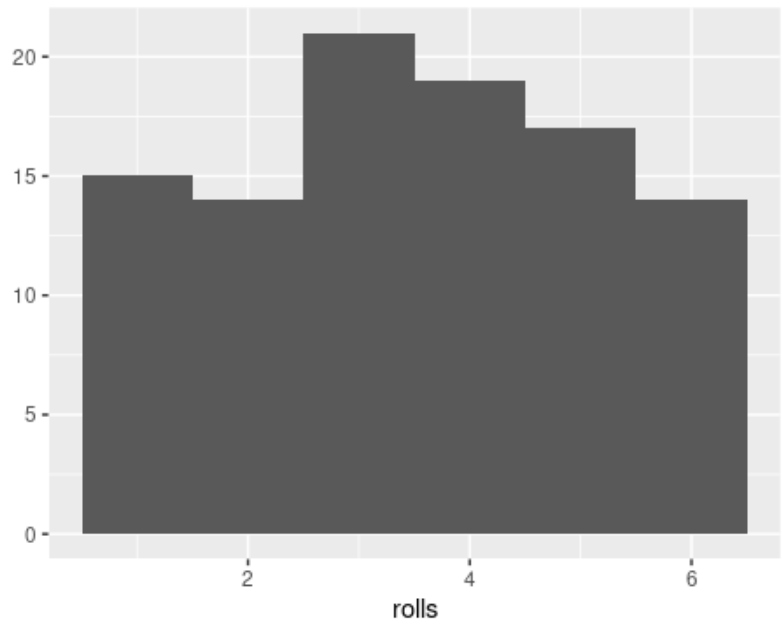


Figure 4: Frequencies of 1, 2, 3, 4, 5, 6 after rolling our virtual die 100 times

It looks like every value occcured roughly 16 times but there is still quite some variation. For instance, the value 3 seems to occur more than 20 times whereas the value 2 occurs less than 15 times. Maybe we have to give it another trial with more replications. With the computer we can do this with a fingertip. Let us roll our die 10000 times and plot the result.

```
> rolls2 <- replicate(10000, roll_die())
> qplot(rolls2m bindwidth = 1)
```

Now the picture looks better.

It seems that there is no significant visual evidence that our die is loaded.

Getting Help

We have now learned a tiny number of R functions and we have written one function ourselves. We have learned how to make use of functions provided by packages. It would be overwhelming to memorize and learn them all. In R, fortunately, every functions comes with a detailed documentation and with its own help page. You need to learn how to use this source right from the beginning.

To access the help page of a function you type the function name preceeded by a questionmark at the prompt, like this

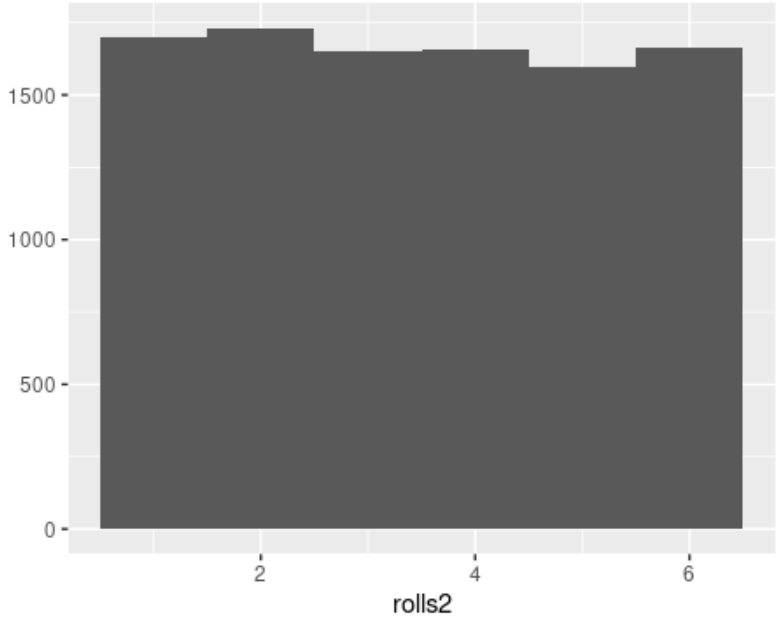


Figure 5: Frequencies of 1, 2, 3, 4, 5, 6 after rolling our virtual die 10000 times

```
> ?sample
```

Then, after you have pressed the return key, a help page is opened in the right lower pane under the help tab. This help page has a particular structure that you will find for every other R function no matter whether it is provided by the base installation or by a package. In the upper left corner you see the name of the function (`sample`) and in curly brackets next to it the term `base`, which means that this is a function in the R base installation. Then you see a headline about what the function does.

From the top of the page, you then first see the field **Description**. This is a short description what the function does. Here it says

```
Description
sample takes a sample of the specified size from the elements of x using either with or without r
```

The next field is **Usage**. It gives you the function description with the arguments. Here for example

```
Usage
sample(x, size, replace = FALSE, prob = NULL)

sample.int(n, size = n, replace = FALSE, prob = NULL,
           useHash = (!replace && is.null(prob) && size <= n/2 && n > 1e7))
```

The first line in Usage should by now be familiar. Don't worry about the second line. The function can obviously do more than we know so far.

Next comes a list of arguments the function takes and what type of information R expects you to provide, as well as what R will do with this information. Here it says for example

```
Arguments
x
either a vector of one or more elements from which to choose, or a positive integer. Se

n
a positive number, the number of items to choose from. See 'Details.'

size
a non-negative integer giving the number of items to choose.

replace
should sampling be with replacement?

prob
a vector of probability weights for obtaining the elements of the vector being sampled

...
```

We omit some information here.

Then comes a field called **Details** which gives a more in-depth description of the function. The next field is calles **Value**. It describes what the function returns when you run it. Then we have a referecne to related R functions under the field **See**. Finally there is a field called **Examples**. This field contains example code that is guaranteed to work. It shows a couple of different cases how you can use the function in practice.

Some more examples

To close this lecture let us come back to probability and discuss some more examples of random experiments, sample spaces, basic outcomes and events. Let us try to leverage our R knowledge to build the objects in the different examples ourselves.

The stock price of Apple

Since the context of this course is Computational Finance, let us discuss an example from Finance first: The stock price of Apple is recorded daily. Let us

use this example to make a first step in loading data into R. We have already learned how we could have a first quick glance at how they look like. We will discuss various ways to get financial data as we go along. For this initial example we have prepared a data file for the Apple stock price from January 2021 to the beginning of September 2021 and we have stored the data (in some format) locally. In practice we often will not have data ready fro retrieval but will have to retrieve them from the web, from a database or from other sources but let us not worry about these practical details for the moment. We have stored the data in an R internal format, called RDS and we have stored the file `apple_stock_prices.RDS` in a folder in our current working directory which we have called `data`. Under this assumption we read these data and assign them to an R object using the R function `readRDS`. Let us call this object `apple_stock`:

```
apple_stock <- readRDS("data/apple_stock_price.RDS")
```

To have a first look at the data, we use the R-function `head()`. This is a very useful function to glance at the data because instead of shwoing us the entire, possibly very big, dataset it shows us just the first few lines. We can even specify how many by specifying a parameter for the number of lines we want to display, say 10:

```
head(apple_stock, n = 10)
```

```
##    symbol       date   open   high    low  close    volume adjusted
## 1    AAPL 2021-01-04 133.52 133.61 126.76 129.41 143301900 128.8048
## 2    AAPL 2021-01-05 128.89 131.74 128.43 131.01  97664900 130.3973
## 3    AAPL 2021-01-06 127.72 131.05 126.38 126.60 155088000 126.0080
## 4    AAPL 2021-01-07 128.36 131.63 127.86 130.92 109578200 130.3078
## 5    AAPL 2021-01-08 132.43 132.63 130.23 132.05 105158200 131.4325
## 6    AAPL 2021-01-11 129.19 130.17 128.50 128.98 100384500 128.3768
## 7    AAPL 2021-01-12 128.50 129.69 126.86 128.80  91951100 128.1977
## 8    AAPL 2021-01-13 128.76 131.45 128.49 130.89  88636800 130.2779
## 9    AAPL 2021-01-14 130.80 131.00 128.76 128.91  90221800 128.3072
## 10   AAPL 2021-01-15 128.78 130.22 127.00 127.14 111598500 126.5454
```

Now we see what is contained in the data. They are organized in a table. Each column is a variable, starting with the stock symbol of Apple, then the date, the opening price, the highest, the lowest and the closing price, the volume and the adjusted volume. Every row is a record of the values of these variables for a particular trading day in the time period we consider.

Now let us make a chart of the closing price at each day over the time period of our data using the plotting function `qplot()` we already used for visualizing the outcomes of repeated die rolls. In order to do so, we create two new R objects which contain the date and the column close. This means we need to select these two columns.

We will learn more systematically how to do this in R. For the moment we just

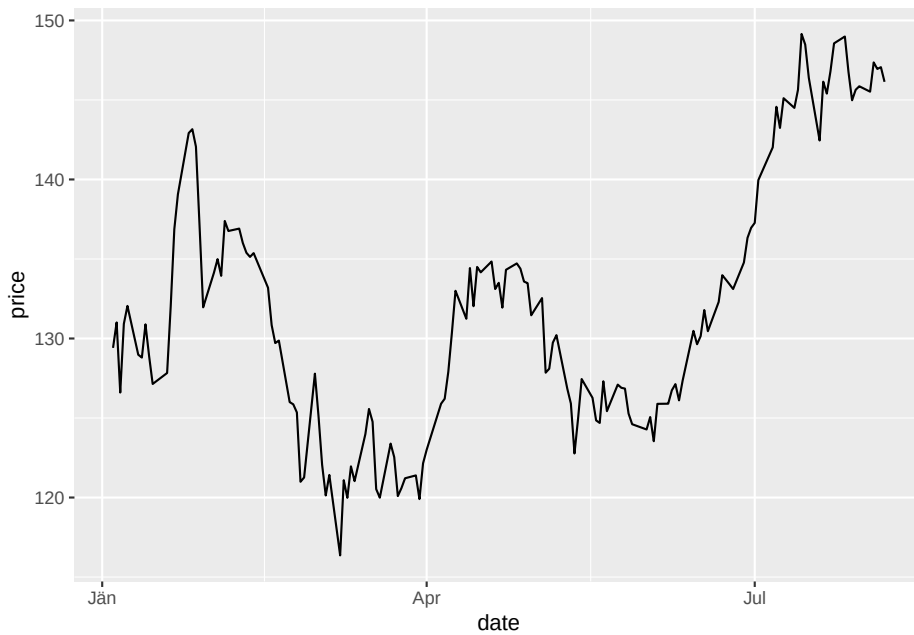
show one way to achieve this for the particular example:

```
date <- apple_stock$date
price <- apple_stock$close
```

Now we load the library `ggplot2` and use the `qplot` function. As arguments we give the qplot function the series of dates and the series of closing prices and set a display option, called `geom` to "line" to get a line graph (instead of the qplot default of a point graph)

```
library("ggplot2")

qplot(date,price, geom = "line")
```



Now assume we are at the end of our price data series, at August 6th 2021. We want to think about the uncertain price tomorrow. This is our random exmeriment. What would be a suitable state space?

What are the possible prices Apple can attain? The minimum price is zero. The maximum price can not be determined. In principle it could be infinite. The state space here would be an open interval of real numbers, $\mathcal{S} = [0, \infty)]$. Quite complicated.

We could define the state space for the price change, in another way, by thinking about three events: Up, denoted by u when the stock price increases, Even, denoted e when the stock price does not move, and Down, denoted d when the price decreases. From this perspective the state space would be $\mathcal{S} = \{u, e, d\}$. A bit simpler than before.

There are more elegant and powerful ways to read and plot data in R. We used this particular example just to show you one particular way. More details and more general insights how to do this we will develop as we go along.

The change of a stock price two days from now

Suppose we want to study the random experiment of observing a stock price move twice. This is a random experiment because whether the coin will go up (U) or down (D) is uncertain before we make the observation, but once the closing price is published, the outcomes are clear. The sample space is $\mathcal{S} = \{UU, UD, DU, DD\}$. The event that the closing price at the end of the first trading day will be higher is $A = \{UU, UD\}$. The event that the stock price does not decrease is $B = \{UU\}$, which is also a basic outcome of the random experiment.

While this example is so simple that we do not need a computer, let us take the opportunity to introduce a new R function, which computes all possible combinations of objects. Let us first create a virtual price move by listing the possible moves at day 1 and day 2.

```
day_1 <- c("U", "D")
day_2 <- c("U", "D")
```

There are some new things here. Let us explain. First, while we used gray boxes to show you input and output of code as it would appear if you interact on the console we use here a very convenient feature in RStudio, which we will use from now on.

We compose these lecture notes in a document format, which allows the mixing of text and code. Whenever we write a piece of R code, it is shown in a gray box, as before but without the `>` prompt. When you move the cursor on the gray box now, you will see a copy icon in the upper right corner of the box. This will allow you to copy this code snippet and use it in another document or on your console to execute it.

The second new thing is that we have used a new data type, which we will need very often: characters. Characters can be used to form strings and R has many powerful functions to work with and operate on strings. We will see many examples of this later. Strings are always between `" "`, like this. Like numbers, strings can be stored in objects and referred to later. But what we need here is actually two strings in one vector. This is achieved by using the `c()` function of R. c stands for concatenation and here it stores both basic outcomes of a price move in two single vectors `day_1` and `day_2`.

We can now pass these two virtual oves to the R function `expand.grid()`. This function will produce the sample space of two price moves by computing all possible combinations of sequences of U and D that can occur at the end of two trading days.

```
expand.grid(day_1, day_2)
```

```
##   Var1 Var2
## 1    U    U
## 2    D    U
## 3    U    D
## 4    D    D
```

Don't worry at the moment if you do not understand what the output format is and why the columns are headed by **Var1** and **Var2** followed by the symbols `<fctr>`. We will come back to this.

Before we look at yet another sample, let us mention that this setup, simple as it may seem, plays a very important role in the theory of random price fluctuations of asset prices and in option pricing and we will encounter it later in more detail.

Boys and Girls

A family has four children. Which one of these children will turn out to be a boy or a girl is uncertain ex ante but once the baby is born we can see whether it is a boy or a girl. Conceptually we can thing of this as a random experiment. Can you figure out the sample space ? Can you figure out the event that the family will have two boys?

Birthdays

Assume there are 10 students in a class. When we assume that the year has 365 days, how many configurations of birthdays are possible? Again we can conceptually think of the birthday of the students in the class as the outcome of a random experiment. The number of basic outcomes are all the possible configurations of 25 birthdays among the 365 days of the year.

Here we encounter a situation where it would be possible but impractical to write down the sample space explicitly. It would just be a very long list of 25 dates long sequences of birthdays. Not very useful. Most of the time sample spaces will be like this. It is therefore practical to know counting techniques to figure out the number of possible outcomes. The example we are considering here is a special case of a general counting problem often encountered in probability: The random placement of `r` balls in `n` cells. The mathematical notation for such a counting problem is: $\binom{n}{r} = \frac{n!}{k!(n-r)!}$ The left hand side of the equation means "the number of ways to choose r things from among n things. The right hand side shows a formular how this can be computed. R has a function to compute these things directly. It is called `choose()` and we apply it to the birthdays example

```
choose(365, 10)
```

```
## [1] 1.021296e+19
```

A pretty large number. Here you see that R displays the number in scientific notation meaning $1.021296 * 10^{19}$, a number with 13 zeros. Pretty large.

Summary

These are the basic **probability concepts** we have covered in this lecture:

1. A **random experiment**: A process leading to an uncertain outcome.
2. **State space**: The collection of all possible outcomes of a random experiment.
3. **Basic outcome**: A possible outcome of a random experiment.
4. **Event**: An event is a subset of basic outcomes. Any event which contains a single outcome is called a simple event.

These are the **R concepts** we have covered in this lecture:

1. **objects** arbitrary names that can store different values and data types.
2. **functions** ab R object that can accept other R objects as arguments, operate on them and return a new object.
3. **scripts** files that store sequences of R commands and can be saved, repopened and allow the execution of commands.
4. **using packages**
5. **finding help**

Project 1: Rolling two dice and adding the points

Here we formulate a project where you can apply the concepts we have learned in this lecture. In this project we ask you to write an R function that virtually throws two dice and adds up the number of points shown by the dice. Using this function do a simulation of this random experiment. Install and load the `ggplot2` package to make a plot using the qplot function to visualize the behavior of our dice. Are the dice fair?

Probability: Basic Definitions and Rules

Probability in theory and applications of probability

Now that we have introduced the concepts of a random experiment, sample space, events and basic outcomes, we can finally proceed to introduce the notion of *probability*. From the preceding lecture it should be clear that we always speak of probability in relation to a given sample space or a conceptual random experiment.

In lecture 1 we said that *probability* is a measure of how likely an event of an experiment is. Probabilities are expressed as numbers. As observed in Feller (p. 19), these numbers are of the same nature as distances in geometry. In the theory we assume they are given to us. We need not assume anything about how they are measured. In actual applications, the determination of probabilities or the application of the theory to observations requires often sophisticated statistical methods. So, while the mathematical as well as the intuitive meaning of probability are clear only as we proceed with the theory we will get a better ability to see how we can apply this concept.

Basic rules of probability

When we talk of probabilities in this lecture we mostly consider probabilities with relation to a *discrete* sample space. The simple sample spaces we considered in lecture 1 contain only a finite number of points. Such sample spaces belong to the discrete sample spaces.

There are also more complicated discrete sample spaces: Think of the random experiment of tossing a coin as often as necessary to see Heads for the first time. We can begin writing down the basic outcomes as: $E_1 = H, E_2 = TH, E_3 = TTH, E_4 = TTTH, \ldots$. We may consider it possible that Head never appears. This event we should perhaps call E_0. In this case, when the basic events can be

arranged into a simple sequence. A sample space is called *discrete* if it contains only finitely many points, or infinitely many points which can be arranged into a simple sequence.

Not all sample spaces are discrete. Except for the technical tools required there is no essential difference between the two cases. In our discussion of probability in this lecture we consider mostly discrete sample spaces.

Given a (discrete) sample space $\mathcal{S}$ the probabilities assigned to events in this sample space must always fulfill three rules:

1. $P(\mathcal{S}) = 1$, where $\mathcal{S}$ is the sample space.
2. For any event $A \in \mathcal{S}$, $0 \leq P(A) \leq 1$. The probability of an event can never be negative or larger than 1.
3. The probability of the union of two events A and B is always smaller or equal to the sum of the probability of these events looked at in isolation: $P(A \cup B) \leq P(A) + P(B)$.

In the theory of probability we use the language of sets and set theory to describe relations among event. We have used one of these relations in rule three, where we expressed the relation between two sets as their union $\cup$. It is useful in studying probability to know a few of these set theoretic definitions.

Let's go through them and illustrate the concepts in the context of the examples we have already developed in lecture 1.

Union The **union of two events** A and B is the set that contains all events that are either in A or in B or in both sets. Set union is written as $A \cup B$.

Let us use the example of the experiment of rolling a die. The sample space $\mathcal{S}$ is the set of all possible outcomes of rolling the dice $\mathcal{S} = \{1, 2, 3, 4, 5, 6\}$. Assume one event is that the outcome is 1, 2 or 3. In set notation, we would write $A = \{1, 2, 3\}$. Let us also assume that the second event is that the outcome is some even number, i.e. 2, 4, 6. Again using set notation we would write $B = \{2, 4, 6\}$. The event $A \cup B$ is then the set of all outcomes such that the outcome is in A or in B or in both or these sets. In such simple examples it is sometimes helpful to depict the situation graphically.

The sample space $\mathcal{S}$ is the gray set containing all possible outcomes of our random experiment. Graphically the union of A and B, $A \cup B$ is a subset of the sample space, the entire colored area.

By the way: R provides functions for computing set operations. Let us use the occasion to show you briefly how to use these functions in the context of this example: We define the sets A and B first using the assignment operator:

```
A <- c(1,2,3)
B <- c(2,4,6)
```

We compute the union by using the function `union()`

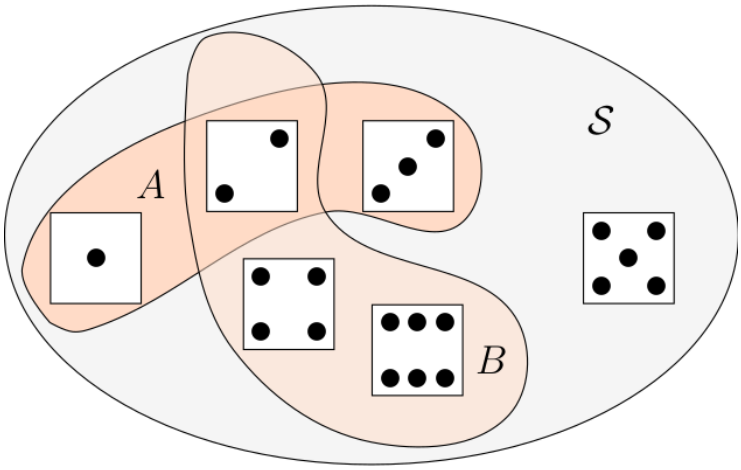


Figure 6: The meaning of set union

```
union(A,B)
```

```
## [1] 1 2 3 4 6
```

which gives us the result we have already derived graphically.

Intersection The **intersection of two events** is the set that contains all events that are both in A and in B. Set intersection is written as $A \cap B$.

We can again illustrate this concept graphically

The intersection of A and B, $A \cap B$ is the orange area containing the dice face with two points. Indeed two is both in A and in B, which is exactly the meaning of set intersection.

The R function for computing intersections is called `intersect()`. We call this function for our example

```
intersect(A,B)
```

```
## [1] 2
```

which gives us the result we have already derived graphically.

Complement The **complement** of an event A with respect to an event B is the set of all elements that are in B but not in A. Set difference is written as $A \setminus B$.

Lets say we want to get the complement, or the set difference of $A \cup B$ with respect to the sample space $\mathcal{S}$. Let us show $\mathcal{S} \setminus (A \cup B)$.

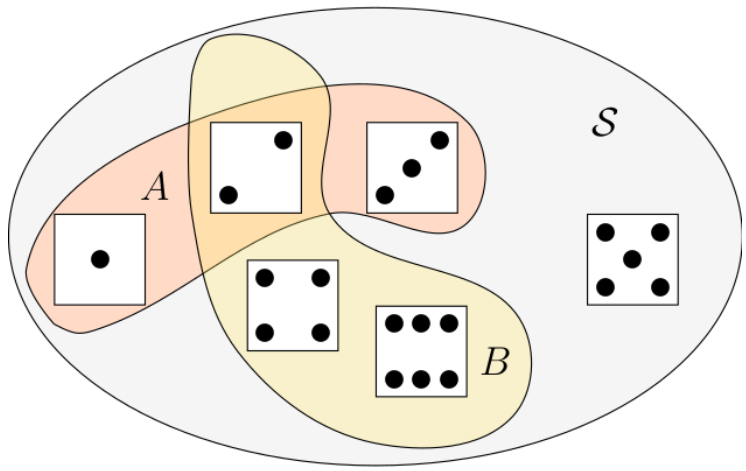


Figure 7: The meaning of set intersection

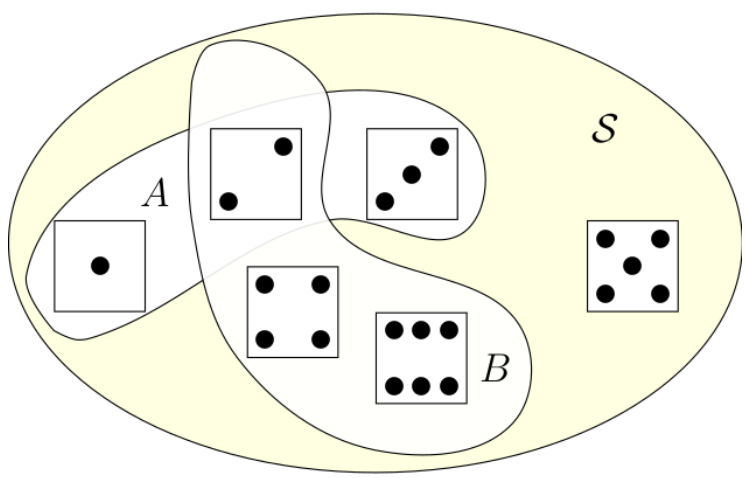


Figure 8: The meaning of set intersection

This complement is the dice shown in the light yellow area, i.e. all the elements of $\mathcal{S}$ which are not in $A \cup B$.

The R function for computing set complements is called `setdiff()`. You can try it with our example:

```
S <- c(1,2,3,4,5,6)

setdiff(S, union(A,B))
```

```
## [1] 5
```

which is 5, as expected.

Mutually Exclusive Two events A and B are said to be **mutually exclusive** if they have no basic outcomes in common. The notation is $A \cap B = \emptyset$

An example in our context is the set of even outcomes $B = \{2, 4, 6\}$ and the set of odd outcomes, let us call it $C = \{1, 3, 5\}$. If we intersect these sets

```
B <- c(2,4,6)
C <- c(1,3,5)

intersect(B,C)
```

```
## numeric(0)
```

we get the empty set, which is expressed by R by giving the data type, in this case numeric, because we are intersecting sets of numeric values, followed by (0). This means, there is no numeric value in the intersection of B and C.

Let us discuss probability rule 3 a bit further: Go to the picture we drew to illustrate the meaning of set union @ref(fig:set-union). To compute the probability $P(A \cup B)$ we have to add the probabilities of all sample points that are contained either in A or in B but each point is to be counted only once. Therefore, probability rule 3, has a (weak) inequality: $P(A \cup B) \leq P(A) + P(B)$.

Now let E be a point contained both in A and in B, in our example this would be $E = \{2\}$, then $P(E)$ occurs twice on the right hand side of our inequality but only once on the left. Therefor the right hand side exceeds the left by the amount $P(A \cap B)$. Thus we have $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

This reasoning holds for arbitrary pairs of events and not only in our example. In the case A and B are arbitrary events and A and B are mutually exclusive, i.e. $A \cap B = \emptyset$ then our inequality in rule 3 becomes an equality and we have $P(A \cup B) = P(A) + P(B)$.

Interpretations of Probability

While rule 1 - 3 pins down three basic mathematical properties of probabilities, there is an ongoing and still unsettled debate what probabilities actually mean.

While in this course we will be less concerned with this philosophical question, it is so important that you should hear about the basic interpretations at least once.

Probability theory took shape in the 17th century through discussions on the mathematics of games of chance, such as poker or roulette or games where dice are rolled. In this stage the *classical* meaning of probability was the ratio of outcomes that satisfy a condition or the description of an event divided by the total number of outcomes in the state space. Of course this interpretation of probability makes the implicit assumption that all outcomes have the same probability. For example, in this interpretation the probability that one role of a dice will result in a 6 is 1/6. This interpretation is often called *classical probability.*

A more important interpretation is the interpretation of probability as *relative frequency.* In this interpretation the probability of an event A is interpreted as the number of times the event A occurs in repeated trials divided by the total number of trials in a random experiment.

A competing interpretation of probability is called *subjective probability.* In this interpretation probability is an opinion or a belief about how likely the occurence of a particular event is.

Example continued

The stock price of Apple

Let us go back to our example with the stock price of Apple. In this example we had expressed the state space as $\mathcal{S} = \{U, D\}$. It is however impossible to determine the probability for each of these two elements. It is also not possible to determine the probability for each price which is theoretically possible.

Tossing a fair coin twice

Assume we toss a fair coin twice and that the two possible outcomes of this random experiment is that the coin comes up heads H or tails T. The state space of the random experiment of tossing a coin twice is $\mathcal{S} = \{HH, HT, TH, TT\}$. We can now ask: What is the probability of getting H after the first toss? Since the coin is assumed to be a fair coin each outcome of the toss is equally likely. The probability of getting H after the first toss, must therefore be 1/2. Now we could ask: What is the probability that we get H after the second toss as well? If you think about the situation, you will see that the probability of getting H in the second toss is completely unaffected by what happened after the first toss. This probability is independent of the outcome of the first toss and since the coin is a fair coin it is also equally likely to get H as it is to get T. The probability of getting H after the first toss is therefore again 1/2.

Here we derived the equal probability postulate by an assumption, interpreting the probability as a classical probability.

We could also give this postulate a frequency interpretation using our knowledge of R.

We proceed by analogy to our approach of rolling a die. Let us therefore create a virtual coin first.

```
coin <- c("H", "T")
```

As in the case of the die we next write a function, so we can virtually toss the coin

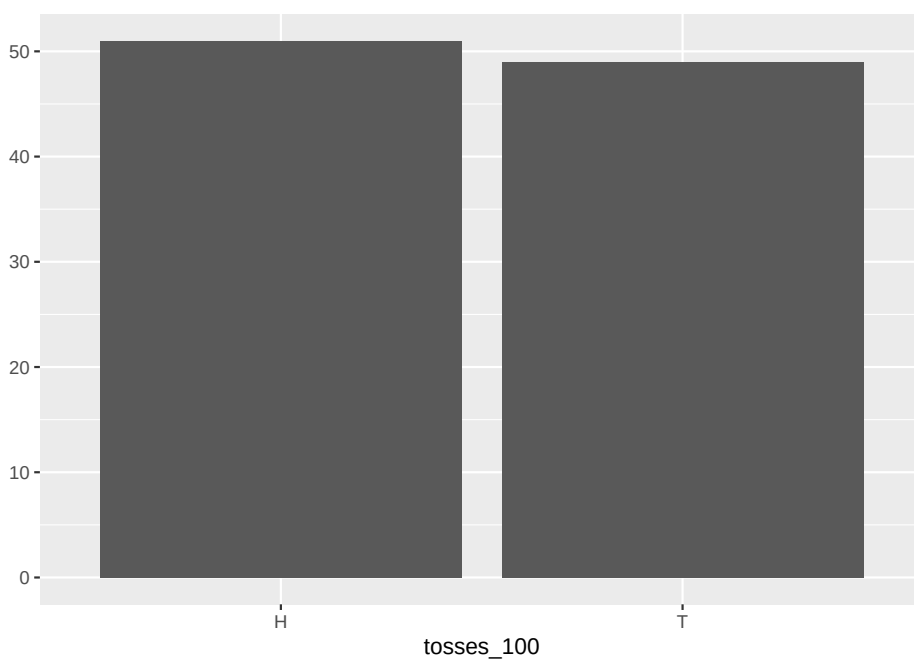
```
toss_coin <- function(){coin <- c("H", "T")
                        sample(coin, size = 1) }
```

Now lets toss this coin 100 times by using the `replicate` function and save the result in an object called tosses_100:

```
tosses_100 <- replicate(100, toss_coin())
```

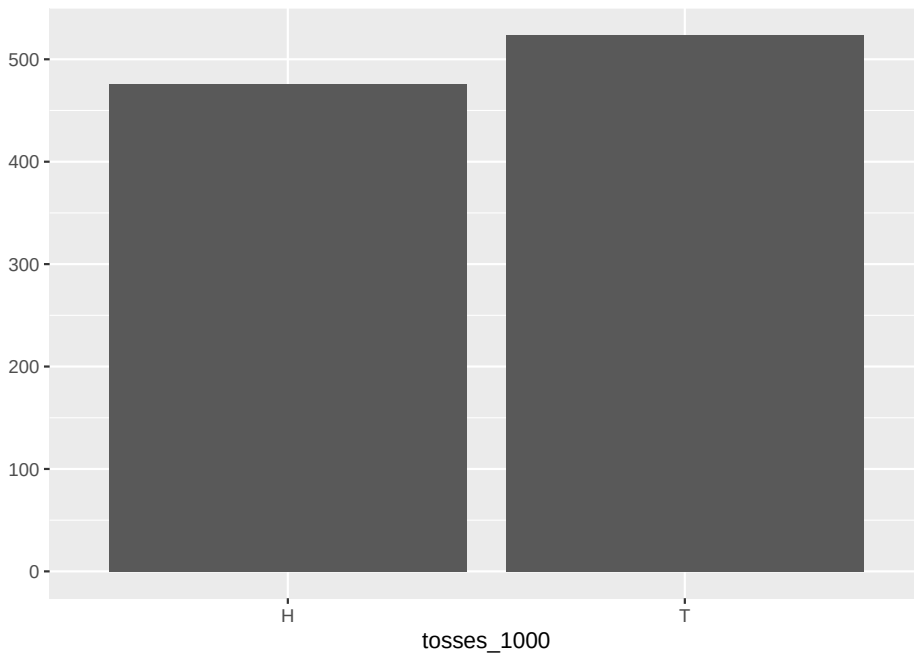
Lets look at the outcome of the 100 tosses graphically

```
qplot(tosses_100)
```



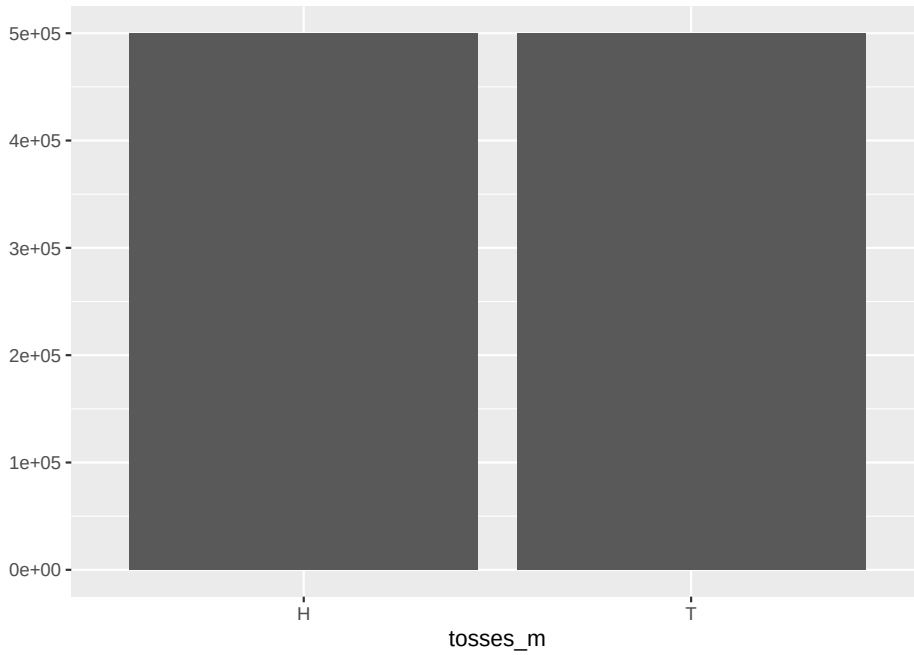
The histogram shows that the occurrence of Heads and Tails are almost 50/50 but not exactly. Lets toss 1000 times and then look again.

```
tosses_1000 <- replicate(1000, toss_coin())
qplot(tosses_1000)
```



This looks already pretty good. Lets toss even more. Why not toss a million of times?

```
tosses_m <- replicate(10^6, toss_coin())
qplot(tosses_m)
```



Now you see the idea of the frequency interpretation. The number of occurrences of H divided by the total number of tosses comes closer to the 1/2 probability as we toss many times. We will come back to this idea again a bit later.

Boys and Girls

We have introduced the four child family before in lecture 1 and asked you to figure out the state space of the random experiment how the sequences of boys and girls might result in this four children outcome.

If you have correctly figured out the state space, you will surely be able to tell which of the following sequences is more likely i.e. has the higher probability: `bbbb`, `bgbg` or `gggg`?

Let's use the idea we considered first in coin tossing, and use the `expand.grid()` function to figure out all combinations of boys and girls in a family of four kids. The outcome of the sex of a baby at any of the four births is abstractly the same as a coin toss with the possible outcomes: `b` or `g`.

```
birth_1 <- c("b", "g")
birth_2 <- c("b", "g")
birth_3 <- c("b", "g")
birth_4 <- c("b", "g")
```

Now we can let R figure out all possible sequences of outcomes. We store the outcome in an object called n and count the combinations using the R function `nrow()` which counts the number of rows of a so called dataframe, a structure

we discuss in detail in a minute.

```
n <- expand.grid(birth_1, birth_2, birth_3, birth_4)
nrow(n)
```

```
## [1] 16
```

There are 16 combinations in total. Applying the same reasoning as with the coin tossing example, we see that the probability of each sequence should be equal to 1/16.

In reality by the way births are slightly more likely for boys than girls. Gelman and Nolan report for example the US numbers for the year 1981, where 1 769 000 girls were born, while there were 1 860 000 boys: 48.7% of births were thus girls.

Let us come to our last example,illustrating the interpretation of probability as subjective probability.

Example: FinTech Start Up

Suppose you had a great idea for a path breaking FinTech innovation, which will disrupt the banking landscape. To predict the success of this innovation in probabilistic terms you could only use a subjective probability. There are no data you can rely on, since this idea had not existed before. You can only rely on your experience and your belief in your idea. There is no way you could use a classical or a relative frequency interpretation of probability.

Independence

Let us go back to our fair die and ask another probability question about the potential outcomes of rolling the die: What is the probability that a fair six sided die will roll a 5 and then a 6?

Note that this is a question, slightly different from what we have asked before. What we need to calculate is actually the probability of the event that the dice shows first 5 and then 6, i.e. we need to compute $P(5 \cap 6)$. We can multiply the individual probabilities because the subsequent rolls of the die are independent: $P(5 \cap 6) = P(5) \cdot P(6) = \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$.

Indeed we have the following definition:

Independence Two events A and B are **independent** if an only if the probability of both A and B occurring is the product of the probability of the two events $P(A \cap B) = P(A) \cdot P(B)$.

Example: The Apple stock price again

Let us go back to our Apple stock price data, we have used previously. We read the data again from our data folder.

```
apple_stock <- readRDS("data/apple_stock_price.RDS")
```

Our data store price information about 150 trading days. During these dates, as you can convince yourself, the price went up on 72 days and it went down on 77 days. It didn't stay the same at any day. We get in total 149 counts because we need to drop one observation to compute the changes in the price.

Now let us assume that the probability of a future up movement is $P(U) = \frac{72}{149}$, which is about 0.48 and the probability of a down movement is $P(D) = \frac{77}{149}$, which is about 0.52.

Now assume in addition that the performance of the Apple stock price on the current trading day is independent of the performance of the Apple stock price on previous trading days. This means that the probability of U-movements and of D-movements is unaffected by the number of previous U and D movements. This is - of course - an assumption. You may think about whether this assumption is reasonable.

Now let us consider a week from Monday to Friday:

- What is the probability that the price of Apple will increase on each of the consecutive days?

There are five trading days, so we need to compute $P(U \cap U \cap U \cap U \cap U \cap U)$. This is by our assumption of independence equal to $P(U) \cdot P(U) \cdot P(U) \cdot P(U) \cdot P(U)$. By our probability estimate from relative frequency this amounts to 0.48^5, which amounts to 0.025.

- What is the probability that the stock price will decrease either on Monday, Tuesday, Wednesday, Thursday or Friday and will increase on the other four days?

The probability that the D movement happens, say on a Monday, is $P(D \cap U \cap U \cap U \cap U)$ or $0.52 * 0.48^4$ which is 0.028. We have in total five mutually exclusive scenarios: $P(D \cap U \cap U \cap U \cap U)$, $P(U \cap D \cap U \cap U \cap U)$, $P(U \cap U \cap D \cap U \cap U)$, $P(U \cap U \cap U \cap D \cap U)$, $P(U \cap U \cap U \cap U \cap D)$. Thus we have $0.028 + 0.028 + 0.028 + 0.028 + 0.028$ as the final probability of this event, which is 0.14.

Analyzing the stock price of Apple: More R concepts

By discussing some probability concepts within the context of stock price movements we looked at the Apple stock price data. R gives us some powerful tools

to work with such data and they provide a very good example to teach you some more R concepts to enhance our toolbox.

Loading Data

We loaded the apple stock price data from a file I had provided you in RDS format. This is not the usual way you will load data into R. The most common case will be that you get a plain text file, usually in a comma separated form or csv for short. In the data directory we have a version of the Apple stock price file we already have already worked with in csv format (`apple_stock_price.csv`). If you open this file in a text editor it will look something like this:

```
symbol,date,open,high,low,close,volume,adjusted
AAPL,2021-01-04,133.520004,133.610001,126.760002,129.410004,143301900,128.804825
AAPL,2021-01-05,128.889999,131.740005,128.429993,131.009995,97664900,130.397324
AAPL,2021-01-06,127.720001,131.050003,126.379997,126.599998,155088000,126.007957
AAPL,2021-01-07,128.360001,131.630005,127.860001,130.919998,109578200,130.307755
AAPL,2021-01-08,132.429993,132.630005,130.229996,132.050003,105158200,131.432465
AAPL,2021-01-11,129.190002,130.169998,128.5,128.979996,100384500,128.376831
AAPL,2021-01-12,128.5,129.690002,126.860001,128.800003,91951100,128.197662 ... etc.
```

Most languages used for data science applications can open plain text files and export data as plain text files. This is why you so often find them. To load a plain text file into R just click the `Import Dataset`icon in R Studios environment pane. Then select "From text file." R-Studio will then ask you to select the file, you want to import. You can use the wizard to tell RStudio what name to give to the data set (for example apple_stock_price). You can also use the wizard to tell RStudio which is the separation character used in your dataset and which character is used to represent decimals (usually . in the US and , in Europe) and whether the data set comes with a row of column names or a so called header. To help you, the wizard shows you what the raw file looks like as well what your loaded data will look like based on the input settings. I recommend to unclick the box "Strings as factors" in the wizard. This will ensure that all character strings are loaded as characters (and not converted tp so called factors). Once everything looks right, click Import. You can examine the data in with the `head()` function.

Sometimes, for instance if you write a script, it is more convenient to use a direct command to read a file. If the decimal sign is `.` (US convention) use `read.csv`, if it is `,` then use `read.csv2()`. The syntax for this function, assuming that the file we want to read is in a directory called data and is named `apple_stock_price.csv` is

```
apple_stock_price <- read.csv("data/apple_stock_price.csv")
```

Please use the help functions to study the details of this function. It is quite powerful. You can inspect the entire dataset using the `View()`function like this:

```
View(apple_stock_price)
```

This command will open a View pane like shown in this picture, which you can use to inspect all the data pretty much like in an excel sheet.

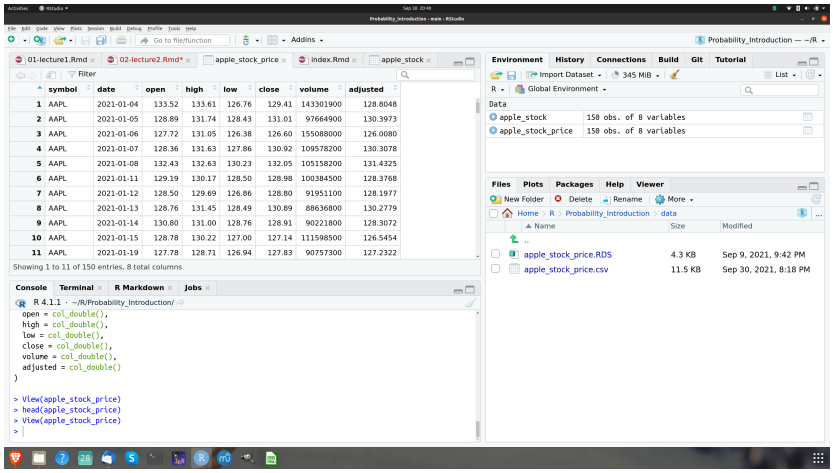


Figure 9: The result of calling the View() function

There are many more possibilities to import data into R from many other formats, which we will not discuss here. The principle is basically always the same and we refer you to the help menus for further details.

Saving Data

Data can be saved by using the `write.csv()` function. It allows you to export data as a comma separated text file. (There are many more export or saving options we do not discuss here). If we would like to save the apple_stock_price object we have just assigned to a csv file we would write

```
write.csv(apple_stock_price, file = "data/prices.csv", row.names = FALSE)
```

Wit this command R turns the R-object `apple_stock_price` into a csv file with the name `prices.csv` and stores it in a subdirectory called data. There are many possible arguments you can set with `write.csv()` (see ?write.csv for details) but there are three you should always use: First you should give `write.csv()` the name of the object you want to save. Then you should give a name (if necessary with a path) to the object. Make sure to give the full name with extension. Finally you should set the argument `row.names = FALSE`. It prevents R from adding an extra column with a counting index for the rows. It is unlikely that another program which reads the data will interpret the row name system correctly.

Atomic vectors

An atomic vector is just a simple vector of data, like the die we have already constructed

```
die <- c(1, 2, 3, 4, 5, 6)
die
```

```
## [1] 1 2 3 4 5 6
```

We can use R to check whether `die`is a vector

```
is.vector(die)
```

```
## [1] TRUE
```

An atomic vector can have only 1 value also. Each atomic vector stores values as a one-dimensional vector and can only store one type of data. Different types of data can be saved by different types of atomic vectors. R recognizes altogether six basic types of atomic vectors: doubles, integers, characters, logical, complex and raw.´

Doubles

A double vector stores regular numbers. In our data these are the columns open, high, low, close, volume and adjusted. In general R will save any number you type as a double. Doubles can be positive negative, large or small and can have digits or not. Ususally it is obvious what type of object you are working with but you can check also.

Let us create a new object close by selecting the column lose‘from apple_stock_price. In R this is done by typing:

```
close <- apple_stock_price$close
```

We use the symbol `$` followed by the name of the column we want to select and assign the selected column to an object called `close`. Now check the type by invoking the `typeof()`function:

```
typeof(close)
```

```
## [1] "double"
```

Integers

Integer vectors store numbers that can be written without commas. We will not need them very often in probability and finance, because you can save integers also as doubles.

Integers can be specifically created in R by typing the number followed by an upper case L.

```
int <- c(-1L, 2L, 4L)
int
```

```
## [1] -1  2  4
```

```
typeof(int)
```

```
## [1] "integer"
```

Characters

A character vector stores small pieces of text. For instance the stock symbol is such a character. Lets select the symbol column and check:

```
symb <- apple_stock_price$symbol
typeof(symb)
```

```
## [1] "character"
```

The individual elements of a character vector are called strings. A string can contain more than just one letter. Character strings can be assembled from numbers and symbols as well. We have encountered characters in some of our examples already before.

It is easy to confude R objects with character strings and you have to be careful to avoid that. For example `symb` is the name of an R object names symb, while "symb" in quotations is a string containing the characzers symb. If you forget the quotation marks R will look for an object which probably won't exist.

Logicals

Locial vectors store TRUE and FALSE, Rs form of boolean values. Logicals are very helpful for comparisons. Assume we are interested how many closing prices are smaller than 124. We have already created an object `close`. So we would type

```
close < 124
```

```
##   [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [37]  TRUE  TRUE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [49]  TRUE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
##  [61]  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [85] FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE
##  [97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
## [109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE FALSE FALSE FALSE FALSE
```

Coercion

R implements a coercion system for types which always follows the same rules. Once you are familiar with these rules you will find them very useful.

How does R coerce data types? If a character string is present in an atomic vector, R coerces every element in this vector to character strings. If the vector contains logicals and numbers every TRUE becomes a 1 and every FALSE becomes a 0. The same coercion rules are used when you try to do math with logicals

This is very convenient. Say we want to count the instances when the closing price of Apple stock was smaller than 124, i.e. when the logical is TRUE. We can use the `sum()`function for that because R will treat each TRUE as a 1 and FALSE as a 0 when used with an algebraic operation.

```
sum(close < 124)
```

```
## [1] 24
```

We count exactly 24 days when this was true in our sample. As a consequence of this cool feature we can compute the porportion of days in our sample when the stock price of Apple was below 124 as

```
mean(close < 124)
```

```
## [1] 0.16
```

Complex and Raw

These data types are much less common than the ones we discussed before. They store complex numbers and raw bytes of data. At the moment we will not say much more about these two types.

Lists

Lists are like atomic vectors because they group data into a one-dimensional set. However lists do not group individual values but they group R objects, such as different atomic vectors or other lists. For example, you can make a list that contains a numeric vector of length 31 in the first element, a character vector of length 1 in its second and a new list with length 2 in its third. This is done by using the `list()`function:

```
test_list <- list(100:130, "R", list(TRUE, FALSE))
test_list
```

```
## [[1]]
##  [1] 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118
## [20] 119 120 121 122 123 124 125 126 127 128 129 130
##
## [[2]]
## [1] "R"
##
## [[3]]
## [[3]][[1]]
## [1] TRUE
##
## [[3]][[2]]
## [1] FALSE
```

`list` creates a list in the same way as `c` creates a vector. Seperate every element in the list by a comma.

The double-bracketed indexes tell you which element of the list is beeing displayed. The single bracketd indexes tell you which subelement of an element is beeing displayed.

Data Frames

Data frames are the two-dimensional versions of a list. They are by far the most frequent data structure we will use. For example the apple_stock_price object we have created by reading the csv datafile is a data frame. Think of a data frame as something like an Excel spreadsheet.

Data frames group vectors together into a two dimensional table. Each vector becomes a column of this table. As a result each column can contain different types of data. The symbol column contains characters, the close column contains doubles for example.

Project 2: Analyzing stock price changes

1. Load the data for the netflix stock price.
2. Estimate the probability that the netflix stock price will rise from the share of daily up moves in the total trading days.
3. Estimate the probability that the stock price will fall in a similar way.
4. Assuming independence what is the probability that the price will rise from Monday to Wednesday and than fall on Thursday and Friday?

Conditional Probability

Appendix

Installing R

Installing RStudio